

Authentication & Authorization Module

Documentation

Overview

This module implements a complete user authentication and authorization system using FastAPI, SQLAlchemy, JWT (JSON Web Token), and Role-Based Access Control (RBAC).

The system provides:

- User Registration (Signup)
- User Login
- Password Hashing
- JWT Token Generation
- Protected API Endpoints
- User Authentication
- Role-Based Authorization (User/Admin)

Project Architecture

Client

|



FastAPI Routes

|



Authentication Layer

|

—— Password Hashing

—— JWT Token Creation

—— JWT Validation

—— Role Verification

|



Database Layer (SQLAlchemy)

PostgreSQL / MySQL / SQLite

JWT Configuration

SECRET_KEY = "..."

ALGORITHM = "HS256"

ACCESS_TOKEN_EXPIRE_MINUTES = 30

Parameter	Description
SECRET_KEY	Used to sign and verify JWT tokens
ALGORITHM	Encryption algorithm used by JWT
ACCESS_TOKEN_EXPIRE_MINUTES	Token validity period

NUTES

Access Token Generation

Function

`create_access_token()`

Purpose

Creates a JWT token after successful user authentication.

Workflow

User Data

|



Add Expiration Time

|



Encode JWT

|



Return Token

Token Payload Example

```
{  
  "sub": "john",  
  "role": "admin",  
  "exp": 1756728000  
}
```

Database Initialization

Code

```
Base.metadata.create_all(bind=engine)
```

Purpose

Creates all database tables defined inside SQLAlchemy models if they do not already exist.

User Registration API

Endpoint

POST /signup

Description

Registers a new user in the system.

Request Body

```
{  
  "username": "john",  
  "email": "john@example.com",  
  "password": "password123",  
  "role": "user"  
}
```

Processing Steps

Step 1: Check Existing User

```
db.query(User).filter(User.username == user.username)
```

Verifies whether the username already exists.

Step 2: Hash Password

```
hash_password(user.password)
```

Converts plain text password into a secure hashed value.

Step 3: Create User

```
new_user = User(...)
```

Creates a new database object.

Step 4: Save User

```
db.add()
```

```
db.commit()
```

Stores the user in the database.

Response

```
{  
  "message": "User registered successfully",  
  "id": 1,  
  "username": "john",  
  "email": "john@example.com",  
  "role": "user"  
}
```

User Login API

Endpoint

POST /login

Description

Authenticates users and returns a JWT access token.

Request Format

username=john

password=password123

Processing Steps

Step 1: Fetch User

`db.query(User)`

Retrieves user record from database.

Step 2: Verify Password

`verify_password()`

Compares provided password with stored hash.

Step 3: Generate JWT

`create_access_token()`

Creates a JWT containing:

- Username
- Role
- Expiration Time

Response

```
{  
  "access_token": "JWT_TOKEN",  
  "token_type": "bearer",  
  "role": "user"  
}
```

OAuth2 Authentication Scheme

Configuration

```
oauth2_scheme = OAuth2PasswordBearer(  
    tokenUrl="login"  
)
```

Purpose

Extracts JWT token from the Authorization header.

Example Header

Authorization: Bearer <JWT_TOKEN>

Current User Authentication

Function

`get_current_user()`

Purpose

Validates JWT token and retrieves authenticated user information.

Workflow

Receive Token

|



Decode JWT

|



Validate Signature

|



Extract User Data

|



Return Current User

Returned Object

```
{  
  "username": "john",  
  "role": "admin"  
}
```

Protected Route

Endpoint

GET /protected

Description

Accessible only to authenticated users.

Authentication Required

Authorization: Bearer <JWT_TOKEN>

Response

```
{
  "message": "Hello, john!",
  "role": "admin"
}
```

Role-Based Access Control (RBAC)

Function

require_roles()

Purpose

Restricts API access based on user roles.

Workflow

Request



Validate JWT

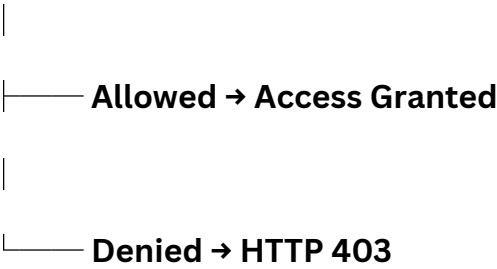


Get User Role





Compare Allowed Roles



Profile Endpoint

Endpoint

GET /profile

Allowed Roles

["user", "admin"]

Access

Role	Access
User	Yes
Admin	Yes

User Dashboard

Endpoint

GET /user/dashboard

Allowed Role

"user"

Response

```
{  
  "message": "Welcome User"  
}
```

Admin Dashboard

Endpoint

GET /admin/dashboard

Allowed Role

"admin"

Response

```
{  
  "message": "Welcome Admin"  
}
```

Complete Authentication Flow

User Registration

|



Store User In Database

|



User Login

|



Verify Password

|

▼

Generate JWT Token

|

▼

Return JWT To Client

|

▼

Client Sends JWT

|

▼

Validate JWT

|

▼

Authenticate User

|

▼

Authorize Role

|

▼

Access Protected APIs

Security Features

- Password Hashing using bcrypt
- JWT Authentication
- Token Expiration
- OAuth2 Security Scheme
- Protected Routes
- Role-Based Access Control
- Unauthorized Access Prevention
- Secure Password Storage

Future Enhancements

- Refresh Tokens
- Email Verification
- Password Reset
- Account Lockout
- Multi-Factor Authentication (MFA)
- Async SQLAlchemy
- Redis Session Management
- API Rate Limiting
- Audit Logging
- Alembic Database Migrations
- Docker Deployment
- CI/CD Pipeline

PROCESS DOCUMENT

Authentication & Authorization Module Documentation

Overview

This module implements a complete user authentication and authorization system using **FastAPI**, **SQLAlchemy**, **JWT (JSON Web Token)**, and **Role-Based Access Control (RBAC)**.

The system provides:

- User Registration (Signup)
- User Login
- Password Hashing
- JWT Token Generation
- Protected API Endpoints
- User Authentication
- Role-Based Authorization (User/Admin)

Project Architecture

Client

|



FastAPI Routes

|



Authentication Layer

|

- └── Password Hashing
- └── JWT Token Creation
- └── JWT Validation
- └── Role Verification



Database Layer (SQLAlchemy)



PostgreSQL / MySQL / SQLite

JWT Configuration

SECRET_KEY = "..."

ALGORITHM = "HS256"

ACCESS_TOKEN_EXPIRE_MINUTES = 30

Parameter	Description
SECRET_KEY	Used to sign and verify JWT tokens
ALGORITHM	Encryption algorithm used by JWT
ACCESS_TOKEN_EXPIRE_MINUTES	Token validity period

Access Token Generation

Function

`create_access_token()`

Purpose

Creates a JWT token after successful user authentication.

Workflow

User Data

|



Add Expiration Time

|



Encode JWT

|



Return Token

Token Payload Example

```
{  
  "sub": "john",  
  "role": "admin",  
  "exp": 1756728000  
}
```

Database Initialization

Code

```
Base.metadata.create_all(bind=engine)
```

Purpose

Creates all database tables defined inside SQLAlchemy models if they do not already exist.

User Registration API

Endpoint

POST /signup

Description

Registers a new user in the system.

Request Body

```
{  
  "username": "john",  
  "email": "john@example.com",  
  "password": "password123",  
  "role": "user"  
}
```

Processing Steps

Step 1: Check Existing User

```
db.query(User).filter(User.username == user.username)
```

Verifies whether the username already exists.

Step 2: Hash Password

```
hash_password(user.password)
```

Converts plain text password into a secure hashed value.

Step 3: Create User

```
new_user = User(...)
```


Creates a new database object.

Step 4: Save User

```
db.add()
```

```
db.commit()
```

Stores the user in the database.

Response

```
{  
  "message": "User registered successfully",  
  "id": 1,  
  "username": "john",  
  "email": "john@example.com",  
  "role": "user"  
}
```

User Login API

Endpoint

POST /login

Description

Authenticates users and returns a JWT access token.

Request Format

username=john

password=password123

Processing Steps

Step 1: Fetch User

```
db.query(User)
```

Retrieves user record from database.

Step 2: Verify Password

```
verify_password()
```

Compares provided password with stored hash.

Step 3: Generate JWT

```
create_access_token()
```

Creates a JWT containing:

- Username
- Role
- Expiration Time

Response

```
{  
  "access_token": "JWT_TOKEN",  
  "token_type": "bearer",  
  "role": "user"  
}
```

OAuth2 Authentication Scheme

Configuration

```
oauth2_scheme = OAuth2PasswordBearer(  
    tokenUrl="login"  
)
```

Purpose

Extracts JWT token from the Authorization header.

Example Header

Authorization: Bearer <JWT_TOKEN>

Current User Authentication

Function

`get_current_user()`

Purpose

Validates JWT token and retrieves authenticated user information.

Workflow

Receive Token

|



Decode JWT

|



Validate Signature

|



Extract User Data

|



Return Current User

Returned Object

```
{  
  "username": "john",  
  "role": "admin"  
}
```

Protected Route

Endpoint

GET /protected

Description

Accessible only to authenticated users.

Authentication Required

Authorization: Bearer <JWT_TOKEN>

Response

```
{
  "message": "Hello, john!",
  "role": "admin"
}
```

Role-Based Access Control (RBAC)

Function

require_roles()

Purpose

Restricts API access based on user roles.

Workflow

Request



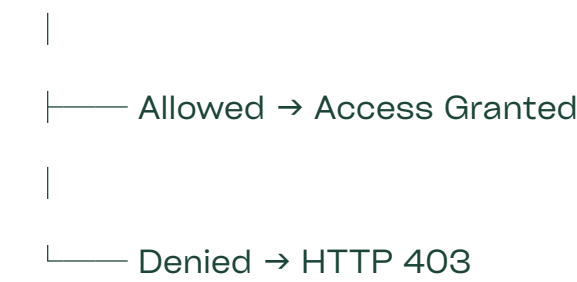
Validate JWT



Get User Role



Compare Allowed Roles



Profile Endpoint

Endpoint

GET /profile

Allowed Roles

["user", "admin"]

Access

Role	Access
User	Yes
Admin	Yes

User Dashboard

Endpoint

GET /user/dashboard

Allowed Role

"user"

Response

```
{  
  "message": "Welcome User"  
}
```

Admin Dashboard

Endpoint

GET /admin/dashboard

Allowed Role

"admin"

Response

```
{  
  "message": "Welcome Admin"  
}
```

Complete Authentication Flow

User Registration

|



Store User In Database

|



User Login

|



Verify Password

|



Generate JWT Token

|



Return JWT To Client

|



Client Sends JWT

|



Validate JWT

|



Authenticate User

|



Authorize Role

|



Access Protected APIs

Security Features

- Password Hashing using bcrypt
- JWT Authentication
- Token Expiration
- OAuth2 Security Scheme
- Protected Routes
- Role-Based Access Control
- Unauthorized Access Prevention
- Secure Password Storage

Future Enhancements

- Refresh Tokens
- Email Verification
- Password Reset
- Account Lockout
- Multi-Factor Authentication (MFA)
- Async SQLAlchemy
- Redis Session Management
- API Rate Limiting
- Audit Logging
- Alembic Database Migrations
- Docker Deployment
- CI/CD Pipeline

Authentication & Authorization Module Documentation

Overview

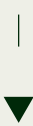
This module implements a complete user authentication and authorization system using FastAPI, SQLAlchemy, JWT (JSON Web Token), and Role-Based Access Control (RBAC).

- Password Hashing
- JWT Token Generation
- Protected API Endpoints
- User Authentication
- Role-Based Authorization (User/Admin)

Project Architecture

Project Architecture

Client

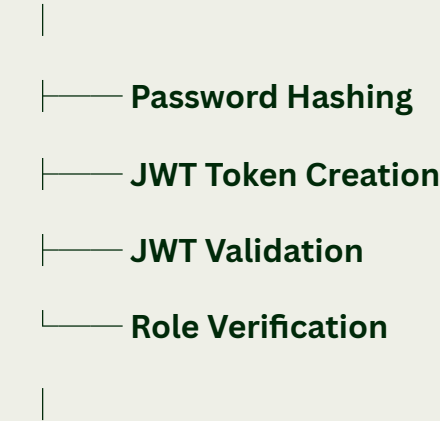


FastAPI Routes





Authentication Layer



Database Layer (SQLAlchemy)



PostgreSQL / MySQL / SQLite

JWT Configuration

SECRET_KEY = "..."

ALGORITHM = "HS256"

ACCESS_TOKEN_EXPIRE_MINUTES = 30

Parameter	Description
SECRET_KEY	Used to sign and verify JWT tokens
ALGORITHM	Encryption algorithm used by JWT
ACCESS_TOKEN_EXPIRE_MINUTES	Token validity period

Access Token Generation

Function

create_access_token()

Purpose

Creates a JWT token after successful user authentication.

Workflow

User Data

|



Add Expiration Time

|



Encode JWT

|



Return Token

Token Payload Example

```
{  
  "sub": "john",  
  "role": "admin",  
  "exp": 1756728000  
}
```

Database Initialization

Code

```
Base.metadata.create_all(bind=engine)
```

Purpose

Creates all database tables defined inside SQLAlchemy models if they do not already exist.

User Registration API

Endpoint

POST /signup

Description

Registers a new user in the system.

Request Body

```
{  
  "username": "john",  
  "email": "john@example.com",  
  "password": "password123",  
  "role": "user"  
}
```

Processing Steps

Step 1: Check Existing User

```
db.query(User).filter(User.username == user.username)
```

Verifies whether the username already exists.

Step 2: Hash Password

```
hash_password(user.password)
```

Converts plain text password into a secure hashed value.

Step 3: Create User

```
new_user = User(...)
```

Creates a new database object.

Step 4: Save User

```
db.add()
```

```
db.commit()
```

Stores the user in the database.

Response

```
{  
  "message": "User registered successfully",  
  "id": 1,  
  "username": "john",  
  "email": "john@example.com",  
  "role": "user"  
}
```

User Login API

Endpoint

POST /login

Description

Authenticates users and returns a JWT access token.

Request Format

username=john

password=password123

Processing Steps

Step 1: Fetch User

db.query(User)

Retrieves user record from database.

Step 2: Verify Password

verify_password()

Compares provided password with stored hash.

Step 3: Generate JWT

create_access_token()

Creates a JWT containing:

- Username
- Role
- Expiration Time

Response

```
{  
  
  "access_token": "JWT_TOKEN",  
  
  "token_type": "bearer",  
  
  "role": "user"  
}
```

OAuth2 Authentication Scheme

Configuration

```
oauth2_scheme = OAuth2PasswordBearer(  
  
    tokenUrl="login"  
)
```

Purpose

Extracts JWT token from the Authorization header.

Example Header

Authorization: Bearer <JWT_TOKEN>

Current User Authentication

Function

```
get_current_user()
```

Purpose

Validates JWT token and retrieves authenticated user information.

Workflow

Receive Token

|



Decode JWT

|



Validate Signature

|



Extract User Data

|



Return Current User

Returned Object

```
{  
  "username": "john",  
  "role": "admin"  
}
```

Protected Route

Endpoint

GET /protected

Description

Accessible only to authenticated users.

Authentication Required

Authorization: Bearer <JWT_TOKEN>

Response

```
{  
  "message": "Hello, john!",  
  "role": "admin"  
}
```

Role-Based Access Control (RBAC)

Function

```
require_roles()
```

Purpose

Restricts API access based on user roles.

Workflow

Request



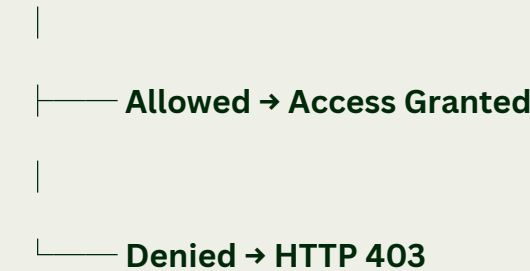
Validate JWT



Get User Role



Compare Allowed Roles



Profile Endpoint

Endpoint

GET /profile

Allowed Roles

["user", "admin"]

Access

Role	Access
User	Yes
Admin	Yes

User Dashboard

Endpoint

GET /user/dashboard

Allowed Role

"user"

Response

```
{
  "message": "Welcome User"
}
```

Admin Dashboard

Endpoint

GET /admin/dashboard

Allowed Role

"admin"

Response

```
{  
  "message": "Welcome Admin"  
}
```

Complete Authentication Flow

User Registration

|



Store User In Database

|



User Login

|



Verify Password

|



Generate JWT Token

|



Return JWT To Client

|



Client Sends JWT

|

▼

Validate JWT

|

▼

Authenticate User

|

▼

Authorize Role

|

▼

Access Protected APIs

Security Features

- Password Hashing using bcrypt
- JWT Authentication
- Token Expiration
- OAuth2 Security Scheme
- Protected Routes
- Role-Based Access Control
- Unauthorized Access Prevention
- Secure Password Storage

Future Enhancements

- Refresh Tokens
- Email Verification
- Password Reset
- Account Lockout
- Multi-Factor Authentication (MFA)
- Async SQLAlchemy
- Redis Session Management
- API Rate Limiting
- Audit Logging
- Alembic Database Migrations
- Docker Deployment
- CI/CD Pipeline

